INSTITUTE OF INFORMATION AND COMMUNICATION TECHNOLOGY

MGM UNIVERSITY, CHHATRAPATI SAMBHAJINAGAR

B. TECH. (INFORMATION TECHNOLOGY)

PROJECT REPORT ON

DocSort

An Intelligent Document Processing and Management System Using OCR and Generative AI

Submitted by

Debashish Kanjalkar | University Exam No.: 202350101114

Abhay Thombre | University Exam No.: 202350101078

Prajwal Joshi | University Exam No.: 202350101084

Rahul More | University Exam No.: 202350101102

Under the Guidance of

Dr. Jaykumar Shreeram Dhage

Academic Year 2026–2027

CERTIFICATE

This is to certify that the following students of B.Tech (Information Technology), Institute of Information and Communication Technology, MGM University, have successfully completed their final-year project titled “DocSort — An Intelligent Document Processing and Management System Using OCR and Generative AI” under the guidance of Dr. Jaykumar Shreeram Dhage during the academic year 2026–2027, in partial fulfilment of the requirements for the award of the Bachelor of Technology degree.

Student name	College Roll No.	University Exam No.
Debashish Kanjalkar	2305123	202350101114
Abhay Thombre	2305085	202350101078
Prajwal Joshi	2305092	202350101084
Rahul More	2305111	202350101102

_____ _____

Dr. Jaykumar Shreeram Dhage
Project Guide

Dr. S. C. Tamane
Director, IICT

Date: _____ Place: Chhatrapati Sambhajinagar

ACKNOWLEDGEMENT

We sincerely thank the Institute of Information and Communication Technology, MGM University, for providing the opportunity and resources to complete our final-year project, DocSort.

We are particularly grateful to our project guide, Dr. Jaykumar Shreeram Dhage, for his guidance and constructive feedback throughout the development process. His suggestions helped us refine the scope of the application and improve the way we approached its implementation.

We also appreciate the support of the faculty, our classmates and our families. This project involved close collaboration among all four members of our group, from understanding the document-management problem to implementation, testing and documentation.

Finally, we acknowledge the open-source developers and documentation teams behind the technologies used in DocSort. Their work made it possible for us to build and examine a practical intelligent document-management workflow.

Submitted by:

Debashish Kanjalkar (Roll No. 2305123) Signature: _____

Abhay Thombre (Roll No. 2305085) Signature: _____

Prajwal Joshi (Roll No. 2305092) Signature: _____

Rahul More (Roll No. 2305111) Signature: _____

CONTENTS

Section	Topic	Page
Preliminary pages		i–vii
Chapter 1	Introduction	1
1.1	Introduction	1
1.2	Problem Statement	1
1.3	Objectives	2
1.4	Scope of the Project	2
Chapter 2	Literature Review	3
2.1	Review of Existing Systems / Related Work	3
2.2	Research Gap	5
2.3	Summary of Findings	5
Chapter 3	System Analysis	6
3.1	Requirement Analysis (Functional and Non-Functional)	6
3.2	Feasibility Study	8
3.3	System Specifications (Hardware/Software)	8
Chapter 4	System Design	9
4.1	System Architecture	9
4.2	Data Flow Diagrams / UML Diagrams	10
4.3	ER Diagram / Database Design	12
4.4	UI/UX Design (Wireframes, Screens)	13
Chapter 5	Implementation	15
5.1	Module-wise Description	15
5.2	Technologies Used	15
5.3	Code Snippets (Important Sections Only)	18
5.4	Screenshots of Outputs	19
Chapter 6	Testing & Validation	21
6.1	Testing Types (Unit, Integration, System)	21
6.2	Test Cases & Results	22
Chapter 7	Results & Discussion	24
7.1	Performance Evaluation	24
7.2	Comparison with Existing Systems	25
7.3	Observations	25
Chapter 8	Conclusion & Future Scope	26
8.1	Summary of Work	26
8.2	Limitations	26
8.3	Future Enhancement	26
	References / Bibliography	27
	Appendices	29

ABSTRACT

Managing digital documents becomes difficult when files are stored with unclear names, scattered across folders, or saved only as scanned images. DocSort was developed to reduce this manual effort by bringing document capture, text recognition, classification, review and retrieval into one application.

The system has a Flutter frontend and a Python FastAPI backend. Tesseract OCR extracts text from scanned files and images; when configured, Google Gemini helps interpret the recognized material and propose document categories and structured fields. Supabase PostgreSQL stores metadata, while private storage retains the original files. Users can correct a predicted category or extracted value before confirming a record in the digital cabinet.

The project initially focused on departmental document handling but evolved toward individual users. Department-related and administrative functions remain in the repository, though they are not presented here as a complete organizational solution. The prototype was tested using Flutter Web and an Android emulator with the backend running locally.

The group manually tested invoices, receipts, certificates, resumes and other PDFs. Most sample documents were processed satisfactorily, although some were misclassified or contained incorrect extracted details. The group corrected those results using the review screen. No labelled benchmark dataset, numerical classification-accuracy score or controlled latency measurement was recorded, so the evaluation is qualitative.

Keywords: document management; Flutter; FastAPI; OCR; Tesseract; Gemini; intelligent document processing; human verification.

ABBREVIATIONS

Abbreviation	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
DFD	Data Flow Diagram
ER	Entity–Relationship
FK	Foreign Key
HITL	Human-in-the-Loop
HTTP	Hypertext Transfer Protocol
JWT	JSON Web Token
OCR	Optical Character Recognition
PDF	Portable Document Format
PK	Primary Key
SQL	Structured Query Language
UI/UX	User Interface / User Experience
UML	Unified Modeling Language
UUID	Universally Unique Identifier

In this report, a verified record means a document that a user has reviewed and explicitly saved. This term does not imply that the AI prediction is always correct.

LIST OF FIGURES AND TABLES

Figure	Description	Page
4.1	Overall system architecture	9
4.2	Context-level DFD	10
4.3	Level 1 document-processing DFD	10
4.4	User use-case model	11
4.5	Document-processing interaction sequence	11
4.6	Key database entity relationships	12
4.7	Login interface	13
4.8	Member dashboard	13
4.9	Scan and upload interface	14
4.10	Processing progress	14
5.1	Review and correction interface	16
5.2	Digital cabinet	16
5.3	Document search	19
5.4	Category chart (cropped)	19
5.5	Export dialog	20
6.1	Validation and manual review flow	23

List of Tables

Table	Description	Page
3.1	Functional requirements	6
3.2	Non-functional requirements	7
3.3	System environment	8
4.1	Selected database entities	12
5.1	Module summary	15
5.2	Selected API routes	18
6.1	Repository validation record	21
6.2	Manual test scenarios	22
7.1	Scope-based comparison	25

CHAPTER 1: INTRODUCTION

1.1 Introduction

A document may be easy to save but difficult to find again. Students keep certificates, resumes, receipts and project material on laptops and phones; families keep bills, identity-related records and scanned forms; small offices store correspondence and reports. In each case, the files often arrive with automatically generated names or are saved in several unrelated folders. Over time, remembering what a file contains becomes more useful than remembering where it was placed.

DocSort grew from this practical observation. The early concept focused on documents handled by departments, but the project team later widened the idea to serve an individual who simply needs a better way to manage personal records. Some administrative and department-related mechanisms remain in the implementation; they are supporting or unfinished functions, not the main claim of the present project. The essential task is to accept a PDF or image, understand enough of its content to suggest a type, and help a person organize and retrieve it.

1.2 Problem Statement

Traditional file management usually relies on filenames, manually created folders and the user's memory. A scanned certificate may be stored as a numbered image, while an invoice may be downloaded with a reference code that gives no clue about the purchase. Opening many files individually to find a date, amount, person or topic wastes time. Manual classification becomes even more difficult when files contain text in images rather than searchable PDF text.

The technical problem addressed by DocSort is therefore not merely file storage. It is the gap between having a digital document and having a usable, searchable record of what that document means. A practical solution must accept everyday PDF and image formats, extract their textual content, propose meaningful metadata, provide understandable organization, and still let the user correct an inaccurate automatic result.

1.3 Objectives

The central objective of DocSort is to reduce routine manual effort in digital document organization while keeping the user responsible for confirming important information. This objective guided the choice of the frontend, backend, processing services and review screen. Rather than building a stand-alone OCR demonstration, the project aims to show how recognized text becomes useful in a document cabinet.

The functional objectives are to provide a common interface for importing PDFs and images; validate incoming files; perform OCR; obtain AI-assisted classification and field proposals when the AI service is available; store original files and document metadata; permit users to correct the extracted document type and fields; and enable navigation through folders, search, analytics and export. The system also preserves a record of the difference between an unreviewed prediction and a confirmed result.

1.4 Scope of the Project

DocSort's present scope is a working academic prototype for general-purpose document management. Its strongest and most clearly demonstrated path is individual use: selecting a sample PDF or image, having it processed, reviewing suggested information, then finding or exporting it later. Supported examples in the group's manual testing included invoices, receipts, certificates, resumes, reports and other PDFs. Both Flutter Web and the Android emulator were used for demonstration, with the FastAPI server running locally.

The system includes additional mechanisms for account approval, departmental organization, administrative views, controlled sharing and an experimental People & Document Map. These features reflect the project's early organizational direction and later development. Their existence in the source code does not mean that the system has been tested or certified as a complete enterprise deployment. The report consequently treats them as secondary implementation areas and identifies the incomplete organizational experience as a limitation.

CHAPTER 2: LITERATURE REVIEW

2.1 Review of Existing Systems / Related Work

Optical Character Recognition is the starting point for many document digitization systems. Smith's description of the Tesseract OCR engine [1] explains how printed document images can be segmented and recognized as text. OCR makes previously image-only content available for later processing, but recognition itself does not determine whether a page is an invoice, a resume or a certificate. A separate interpretation stage is usually needed to give meaning to the recognized characters.

The problem becomes harder when important content is arranged in tables, headers, stamps or forms. The FUNSD dataset introduced by Jaume, Ekenel and Thiran [2] highlights the challenge of understanding noisy scanned forms. It includes annotated entities and relationships, reminding us that a word's position and association with nearby labels can matter as much as the word itself. This observation is relevant to DocSort because extracted field names and values must be checked together.

LayoutLM, proposed by Xu and colleagues [3], extended document understanding by modelling text along with layout information. Their work shows why treating a document as a plain stream of words can miss visual relationships. DocSort does not implement LayoutLM, and it should not be compared to the published model by accuracy. The useful lesson is conceptual: document understanding benefits from more than character recognition alone.

Research has continued beyond conventional OCR engines. TrOCR, described by Li and colleagues [4], uses pretrained Transformer components for text recognition. Its reported research results illustrate how learning-based methods can improve reading of printed or handwritten text in particular benchmark settings. DocSort uses Tesseract rather than TrOCR; the paper is included to explain how OCR technology has evolved, not to imply that those benchmark results apply to this application.

Related Research and Software Systems

Kim and colleagues introduced Donut, an OCR-free document-understanding Transformer [5]. Instead of relying on a separate OCR engine, their method interprets document images in a more unified pipeline. Their paper also discusses how recognition errors can affect later document analysis. DocSort follows the more modular OCR-plus-generative-AI approach, which is easier for the team to integrate and inspect, but it can still inherit mistakes from the first stage.

These different research directions suggest a useful distinction. Recognizing characters answers “What text seems to be present?”; classifying a document answers “What kind of record is this?”; field extraction answers “Which values belong to which labels?” A complete document management workflow then adds another question: “Can the user trust and retrieve this record later?” DocSort addresses all four questions in sequence, but it deliberately keeps the final judgement with the reviewer.

Existing document-management approaches range from ordinary filesystem folders to feature-rich cloud services. A traditional folder system is inexpensive and familiar, but it usually depends on the person creating descriptive names and categories. Standard search can help when the contents are already indexed, yet image-based documents often need OCR before their text is useful. This makes basic file storage only one part of the wider document-management problem.

Commercial platforms illustrate different priorities. Microsoft SharePoint provides document libraries, metadata and search within organizational collaboration environments [7]. Google Cloud Document AI provides managed document-processing services and specialized processors [8]. These platforms serve legitimate needs at different scales, but they are not equivalent to a small local academic prototype. Their available features, deployment assumptions and costs vary, and this report does not claim that DocSort outperforms them.

2.2 Research Gap

The reviewed research often concentrates on a measurable component: OCR accuracy, form understanding, layout representation or end-to-end document interpretation. These are important technical problems, but a user ultimately needs a record that can be corrected, organized and found again. For a student-built general-purpose application, the practical gap is the experience between recognizing a page and reliably managing the document throughout its life cycle.

DocSort responds to this gap with several connected features. It accepts common file types through a Flutter client, uses a backend-controlled processing pipeline, retains the original source in private storage, and presents extracted fields in an editable review interface. It also keeps OCR confidence, suggested filing confidence and human verification state separate. This is important: a high processing score is not a substitute for an actual user checking that the date, name or document category is correct.

The human role is supported by research on machine-assisted annotation. The NIST technical note by Fung and colleagues [6] examines systems in which machine assistance is paired with human work on technical documents. DocSort is not the same system and does not reproduce its studies, but the broader principle is relevant: human oversight can be built directly into an automated interpretation process rather than added only after a mistake is discovered.

2.3 Summary of Findings

The literature review suggests that OCR, document understanding and document management should be treated as related but distinct tasks. Tesseract [1] represents a practical OCR foundation. FUNSD [2] and LayoutLM [3] emphasize the difficulties created by document structure. TrOCR [4] and Donut [5] demonstrate different research approaches to recognition and visual interpretation. Work on machine-assisted annotation [6] provides a useful rationale for keeping a person involved in decisions that affect a saved record.

CHAPTER 3: SYSTEM ANALYSIS

3.1 Requirement Analysis (Functional and Non-Functional)

Requirements were identified from the planned document workflow and refined as the project changed from a department-oriented idea to a primarily individual-user application. The current requirements focus on the actions the user must complete reliably: access the application, submit a file, understand the processing state, inspect extracted information, correct it and retrieve the saved record. The codebase also contains supporting functions for administration, controlled sharing and relationship mapping.

A notable requirement is that an automatic prediction must not silently become a verified record. Every newly processed document enters a “Needs Review” state. Users can edit the type and extracted fields, and the backend saves the confirmed changes. The other main requirement is a coherent file-to-record link: the original file must be available through protected storage while searchable metadata stays in the database.

ID	Functional requirement	Source-supported behavior
FR1	Register and authenticate	Login and account records; new accounts may require approval
FR2	Upload or scan	PDF and image selection, validation and tracked jobs
FR3	Extract and classify	Tesseract OCR and optional Gemini field proposals
FR4	Review and correct	Edit fields and category before confirming
FR5	Organize and retrieve	Cabinet, folders, search and export

Table 3.1: Main functional requirements.

Non-Functional Requirements

Beyond features, DocSort needed to be understandable, responsive during a long-running upload and careful with stored files. The Flutter interface uses familiar navigation elements and gives feedback at each processing stage. The upload endpoint checks file format and size before starting costly processing. A bounded concurrency mechanism limits the number of simultaneously active document processors. These design choices support predictable behavior during a classroom demonstration, although no independent usability study or load benchmark was completed.

Privacy and access control are especially relevant because even sample documents may contain names, addresses or other personal details. The backend owns privileged database and storage access; clients authenticate through JWTs and protected routes check current account state. Passwords are hashed rather than saved as plaintext. These mechanisms are concrete safeguards but should not be confused with a formal security audit or compliance certificate [9], [10].

Requirement	Design response	Known limit
Usability	Clear upload, review and cabinet screens	Mobile analytics layout needs refinement
Privacy	JWT-based endpoints and private source storage	Production security audit not performed
Reliability	File checks, upload-job statuses, review option	Background tasks depend on local server process
Portability	Flutter Web and Android build	Confirmed tests: browser and emulator
Maintainability	Separate Flutter features and FastAPI modules	Future changes require regression checks

Table 3.2: Non-functional requirements and boundaries.

3.2 Feasibility Study

Technical feasibility was assessed primarily through building and running the system rather than through a large formal study. Flutter supplied a shared client codebase for Web and Android; FastAPI exposed HTTP endpoints for the interface; Supabase offered a managed PostgreSQL database and private Storage [12]; and Tesseract provided a local OCR engine. Gemini added structured proposals [14] without the team needing to train a specialized machine-learning model. Flutter's official documentation describes the framework's multi-platform development model [11].

Operationally, the group ran FastAPI and Flutter Web on a local computer and accessed the interface through a browser. The Android emulator was used to test the mobile target. The OCR component therefore needed to be installed and reachable from the computer hosting the backend. Supabase and Gemini required network access and working credentials. This setup was feasible for development and demonstration, but it is different from deploying a reliable public service for many concurrent users.

3.3 System Specifications (Hardware/Software)

The tested system was not hosted as a finished cloud application. In the team's actual setup, a development computer ran the FastAPI backend and the Flutter Web client was opened locally in a browser. The Android emulator acted as a second frontend test environment. Supabase PostgreSQL and private Storage, as well as the configured Gemini service, were external integrations reached through the backend. Tesseract ran as part of the backend document-processing environment.

Layer	Development / test environment	Role
Frontend	Flutter and Dart; browser and Android emulator	User interface
Backend	Python, FastAPI on a local computer	Validation and processing API
Recognition	Tesseract, PyMuPDF and Pillow	OCR and file handling
AI	Configured Gemini API	Optional classification / extraction assistance
Data services	Supabase PostgreSQL and private storage	Metadata and original files

Table 3.3: Development and test environment.

CHAPTER 4: SYSTEM DESIGN

4.1 System Architecture

DocSort uses a client–server arrangement. The Flutter frontend is responsible for displaying screens, collecting user actions and sending authenticated requests. The FastAPI backend is responsible for decisions that should not be trusted to the client: file admission, account and document authorization, processing orchestration, database updates and protected exports. Supabase stores both document metadata and the original bytes, while Tesseract and Gemini provide content interpretation.

Figure 4.1 shows the main deployment connections. The backend was run on the group's computer during testing, not on a public production server. This matters because the browser and Android emulator could reach a local API while the backend itself called remote services. The diagram also keeps OCR local to the backend and places Gemini behind the API boundary rather than in the Flutter client.

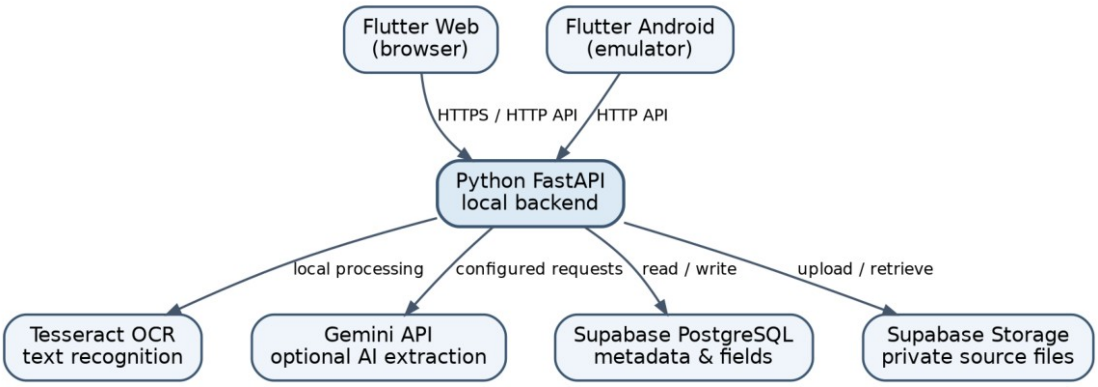


Figure 4.1: Overall DocSort architecture and development connections

FastAPI's background-task approach is convenient for a prototype because the upload request can return a job identifier before OCR finishes [13]. In DocSort, a process-local concurrency limit also reduces uncontrolled simultaneous work. The trade-off is that such tasks are not crash-durable. This limitation is documented in the system and addressed as future scope, not hidden by the architecture diagram.

4.2 Data Flow Diagrams / UML Diagrams

A context-level data flow diagram treats DocSort as one processing system. The ordinary user supplies account information, documents, review corrections and search queries. The application returns progress, extracted fields, organized records and exports. The wider codebase includes administrator approval and oversight routes, but this context-level diagram focuses on the individual-user workflow.

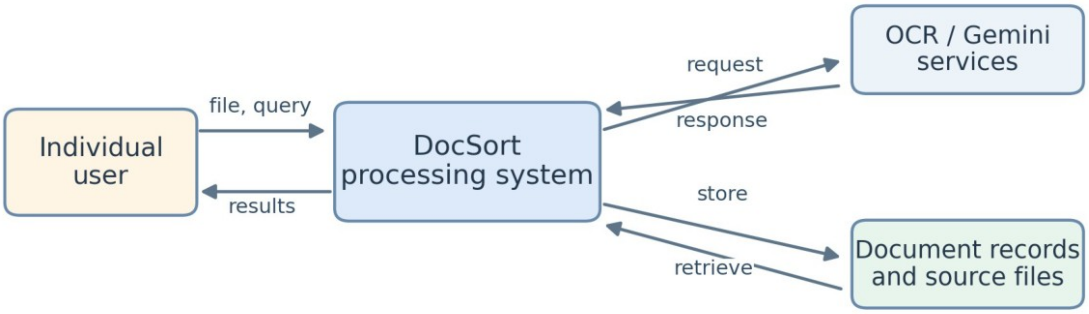


Figure 4.2: Context-level data flow diagram (DFD Level 0)

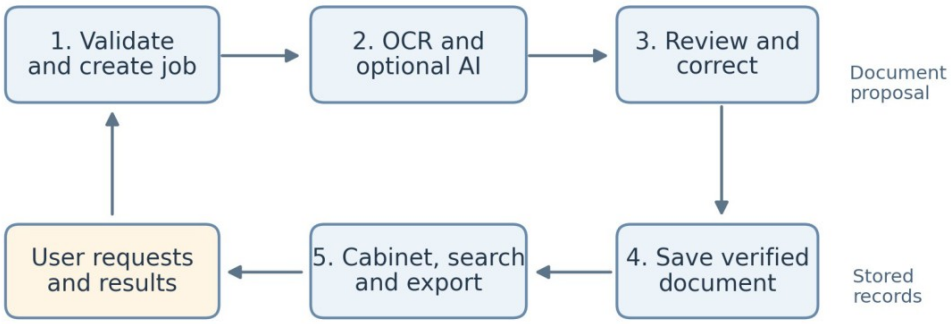


Figure 4.3: Document-processing data flow (DFD Level 1)

Use Cases and Processing Sequence

The use-case model in Figure 4.4 represents actions available through the primary interface. A member can log in, select a document, review extracted information, browse cabinet folders, search and export. Administrative account approval is separated from these ordinary actions. The figure illustrates roles, not a claim that every department-oriented capability was exercised in the group's manual tests.

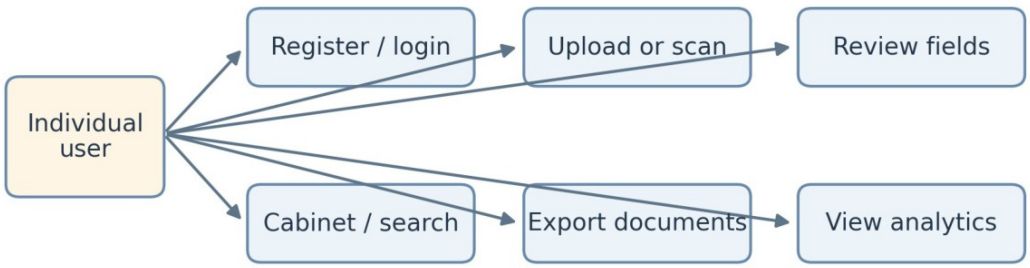


Figure 4.4: Primary user actions in the DocSort application

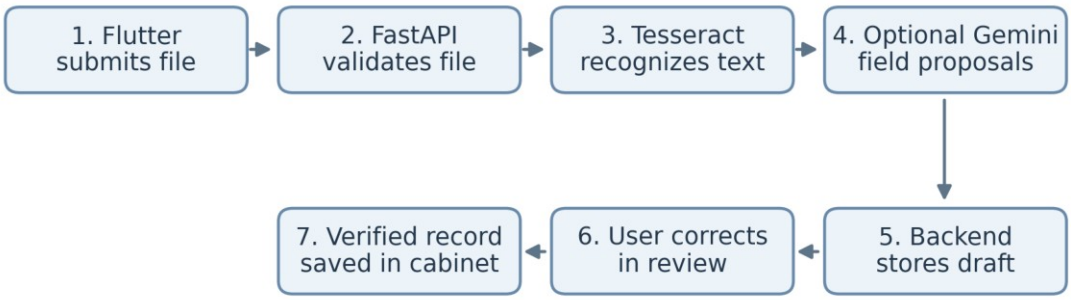


Figure 4.5: Simplified upload-to-verification interaction sequence

4.3 ER Diagram / Database Design

The schema separates users, document jobs, source-document records, extracted fields and organizational folders. A document belongs to an owning user, may be associated with a processing job and folder, and can have multiple extracted field rows. A shared link points to one document and records the information needed to manage access and expiration. Activity records support limited auditing of important events.

Figure 4.6 shows the central tables and foreign-key directions from the supplied SQL schema. The schema also includes additional administrative and People Map entities, which are outside the main ER diagram to keep it readable. A complete schema is more extensive than a single figure; Table 4.1 below summarizes the selected entities and their functions.

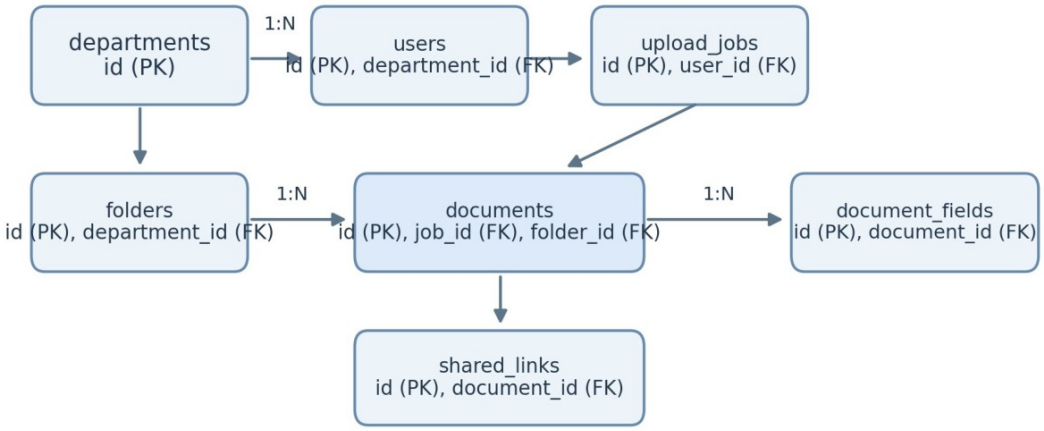


Figure 4.6: Key relationships in the supplied PostgreSQL schema

Entity	Purpose
users / departments	Account and optional organizational association
upload_jobs / documents	Processing state and saved document metadata
document_fields / folders	Extracted values and document organization
shared_links	Optional controlled sharing metadata

Table 4.1: Selected database entities.

4.4 UI/UX Design (Wireframes, Screens)

The interface is organized around a simple journey. The login screen establishes the user session, after which the member dashboard presents the main navigation actions. The dashboard provides visible shortcuts for scanning, opening the cabinet, searching and viewing analytics. The design makes the document lifecycle discoverable without asking the user to understand the technical difference between the OCR service, the AI provider and the database.

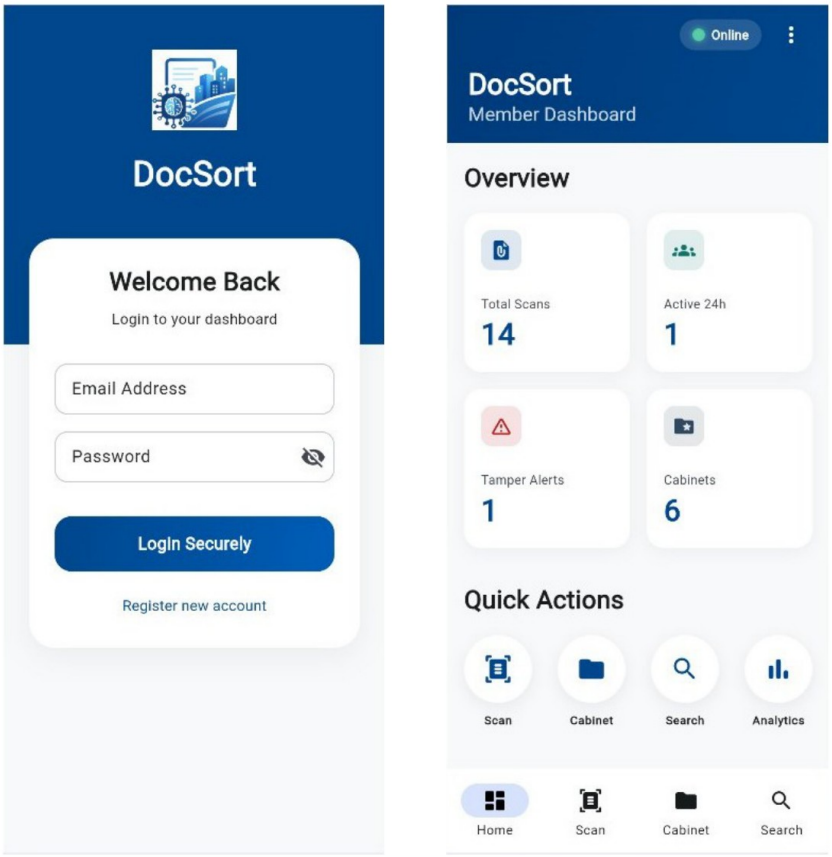


Figure 4.7: Login interface (left); Figure 4.8: Member dashboard (right)

Document Intake and Feedback Screens

The screenshots below were captured from Flutter Web in Chrome using mobile-device emulation, at a 360 × 800 viewport. They are real screens supplied by the project team, not newly drawn mockups. The mobile layout is relevant because DocSort also targets Android. Figure 4.2 earlier in this chapter summarizes the main interactions with the processing system; Figures 4.9 and 4.10 show the upload and processing screens as they appeared in the test application.

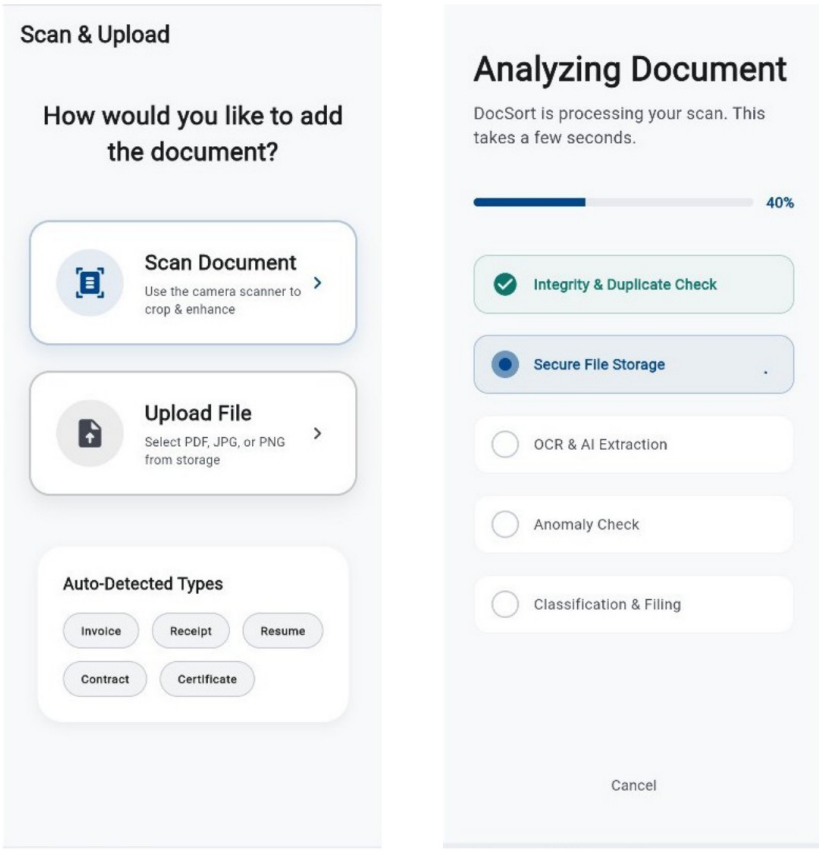


Figure 4.9: Scan and upload screen (left); Figure 4.10: Processing progress (right)

CHAPTER 5: IMPLEMENTATION

5.1 Module-wise Description

Implementation was divided into a Flutter interface and a Python API. The Flutter project groups related screens, providers and services by feature. The backend similarly separates routes, core security and database helpers, processing services and schemas. This structure made it possible to develop the user journey one module at a time and then connect the modules through API requests.

The central workflow is not a single endpoint. Upload first validates a PDF or image and creates a tracked job. Processing happens asynchronously from the user's perspective, and the UI reads the job status. After completion, the user enters a review screen, checks the extracted type and fields, and confirms the record. Cabinet and search then operate on the persistent result. The following pages explain these responsibilities, with short source excerpts and screenshots.

Module	Main implementation responsibility
Authentication	Login, registration and guarded navigation
Upload / processing	File validation, job tracking, OCR / AI orchestration
Review	Correction of predicted type and extracted fields
Cabinet / search	Organized documents, filters, retrieval and export
Analytics	Summary statistics and category charts

Table 5.1: Core implemented modules.

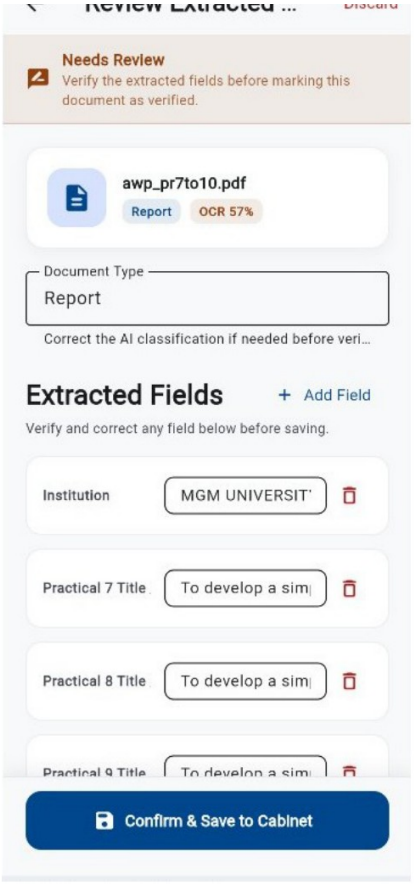
5.2 Technologies Used

Flutter and Dart provide the presentation layer. Riverpod holds and updates application state, GoRouter coordinates navigation, Dio sends requests and FL Chart renders analytics graphics. The repository includes conditional source files for platform-specific scanning and image preview, allowing a Web build to avoid importing mobile-only libraries. Flutter's multi-platform design was helpful because the team could demonstrate a web version and an Android emulator with closely related UI code [11].

Authentication, Human Review and Cabinet

The Flutter authentication screens collect account details and pass them to the FastAPI authentication routes. Login checks the submitted password against a stored bcrypt hash and returns a signed JWT when the account is eligible. The backend also checks the current account record during protected requests instead of relying only on claims kept in the client. This reduces the chance that a previously approved session retains access after an administrator changes the account state.

The review page displays the AI-proposed document category and extracted fields. Users can correct these values before confirming the record. The cabinet then organizes saved documents and separates items that still require review.



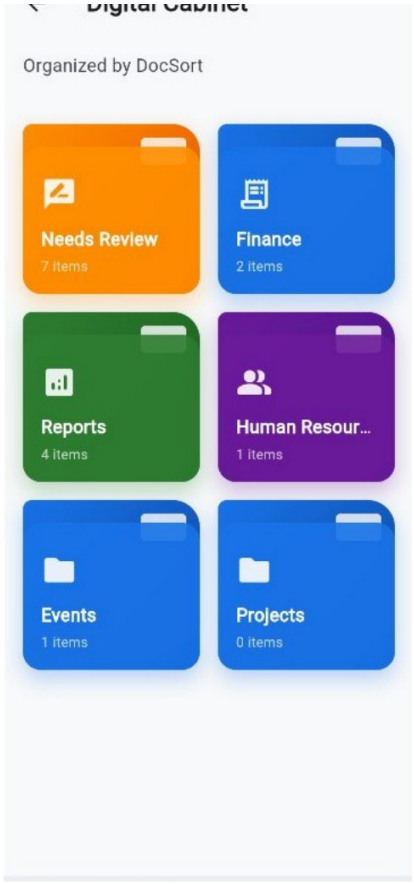


Figure 5.1: Review and correction form (left); Figure 5.2: Digital cabinet (right)

Document Intake and Processing

The Scan & Upload screen offers two entry choices. On supported mobile platforms, a scanner integration can capture a page; users can also select a supported PDF, JPG or PNG file. For Flutter Web, the code uses bytes from the file picker because a browser-selected file is not a normal local filesystem path available to the application. This platform difference is handled in the client implementation rather than exposed as a separate backend workflow.

Once a file is submitted, FastAPI checks its filename, extension, real content signature and parser-readable structure, along with configured size and page limits. An accepted file receives an upload-job identifier. The processing interface can then display which stage is underway: integrity check, source storage, OCR/AI extraction, anomaly analysis and classification or filing. If processing fails, the job records the failure and the user can retry or return to the upload screen.

Tesseract is DocSort's text-recognition engine. For a PDF, PyMuPDF renders pages into images for OCR; for an uploaded JPG or PNG, the input image can be processed directly. The backend does not merely read the first page of an accepted multi-page PDF: it iterates through the accepted pages and combines their recognized text. Per-word confidence values are normalized into an OCR-confidence signal. The value indicates the OCR engine's assessment of its output, not its measured agreement with an independent transcription.

Review and Filing Logic

The team observed during its sample testing that some document types were assigned incorrectly and some fields were extracted inaccurately. Members were able to correct such results. This finding supports the practical value of the review screen, although it does not by itself establish a numerical improvement in extraction accuracy. The corrected values are saved for the particular document; no automatic retraining loop was identified in the submitted implementation.

5.3 Code Snippets (Important Sections Only)

The following excerpts come from app/services/autosort_service.py in the uploaded backend. They show how familiar document labels are mapped to folder names when the workflow needs a deterministic fallback. The mapping is intentionally understandable: an invoice or receipt maps to Finance, while a resume maps to Human Resources and a report maps to Reports. This helps explain why the Cabinet screen can contain category-oriented folders even though the project now targets individual users.

```
DOC_TYPE_TO_FOLDER = {
   "invoice": "Finance",
   "receipt": "Finance",
   "resume": "Human Resources",
   "certificate": "Certificates",
   "report": "Reports",
}
```

Code and API-Level Integration

The API inventory shows a clear division of responsibility. POST /upload admits a document and returns an upload-job response, while GET /upload/status/{job_id} supplies progress and completion information. The document review path is implemented through the cabinet endpoints, including the route for replacing extracted fields. Other routes handle account login, search, analytics, folder exports and secondary functions.

Selected route	Purpose
POST /upload	Accept validated document and create upload job
GET /upload/status/{job_id}	Retrieve processing status
PUT /cabinet/documents/{doc_id}/fields	Update reviewed extracted fields
GET /cabinet/documents	List accessible saved documents
GET /search	Search saved document records

Table 5.2: Selected FastAPI routes, excluding any mounting prefix.

5.4 Screenshots of Outputs

The search screen displays files matching a query, including the source filename, a detected type and a review state. In the captured sample, some records are verified while another remains in Needs Review. This difference is useful evidence that the interface does not flatten all automatic predictions into the same final status.

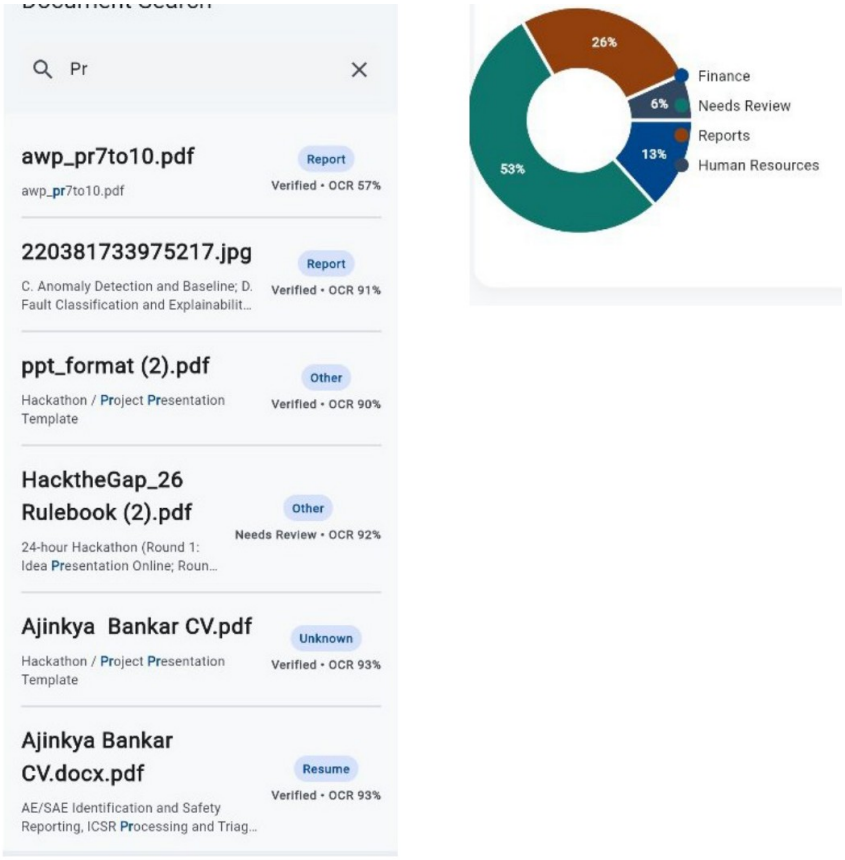


Figure 5.3: Search results (left); Figure 5.4: Category chart (cropped, right)

The analytics figure is cropped from the original screen. A layout overflow in the original summary-card area remains a known mobile-viewport limitation, not a corrected application feature.

Export and Navigation

The document cabinet supports exporting folder content. The supplied screenshot shows the export dialog with CSV and ZIP choices. CSV is useful when a user needs a structured list, while a ZIP archive can collect allowed source documents. The backend includes safeguards around filenames, CSV content, source-file integrity and bounded ZIP creation. The group's manual testing confirmed export worked, although it did not record a timed throughput benchmark or a systematic large-archive evaluation.

Beyond the visible core features, the codebase includes share-link and administration routes. The group could not confirm that sharing and all administrative features were tested in its manual demonstration. They are therefore described as implemented or partially developed code areas rather than as fully validated outputs. A report screenshot is evidence that a screen was displayed; it is not evidence that every possible backend path has been executed.

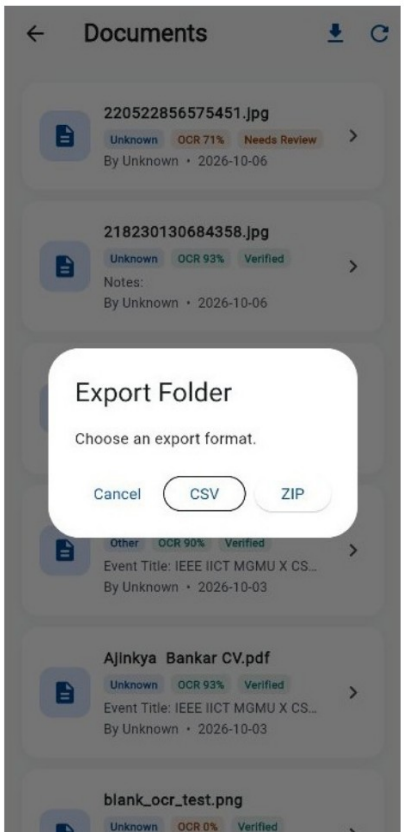


Figure 5.5: Export folder dialog with CSV and ZIP choices

CHAPTER 6: TESTING & VALIDATION

6.1 Testing Types (Unit, Integration, System)

Testing was treated as two different kinds of evidence. The repository includes automated backend and Flutter checks that exercise small units and feature contracts. Separately, the project team manually opened the application and tried real sample documents through the user interface. Keeping these sources apart is necessary because a passing unit test does not prove the full browser-to-cloud path worked, while an observed successful demonstration does not measure all possible edge cases.

Unit-level tests in the backend cover utility behavior such as file safety, confidence handling, authentication checks, autosort mappings and evaluation calculations. Integration-oriented tests exercise API contracts, record access boundaries, upload job state, field replacement and sharing behavior with test fixtures or substitute dependencies where appropriate. Flutter tests cover aspects of navigation, providers and the visual workflow. These are appropriate forms of regression evidence for an academic application.

System-level observations came from the actual Web and Android-emulator demonstrations. The team ran FastAPI locally, uploaded several kinds of PDFs and images, inspected classification and extracted information, corrected inaccurate fields, and checked the cabinet, search, dashboard and export. No formal usability survey, labelled accuracy dataset or independent load test was performed. The report therefore does not convert these observations into numerical OCR or AI performance claims.

Repository validation	Recorded result	Interpretation
Backend automated tests	432 passed (repository release notes)	Documented; not rerun for this report
Flutter automated tests	33 passed (repository release notes)	Documented; not rerun for this report
Flutter Web demo	Working (group confirmation)	Manual functional observation
Android emulator demo	Working (group confirmation)	Manual functional observation

Table 6.1: Sources and limits of the reported validation evidence.

6.2 Test Cases & Results

The group tried several sample file categories, including invoices, receipts, certificates, resumes and reports. The purpose was to see whether the workflow could process a useful variety of ordinary files, not to calculate a statistically representative score. The team reported that most documents were handled correctly and that some required correction. Occasionally a category or extracted value was wrong or missing. The Review screen allowed the group to fix the information and proceed.

Table 6.2 uses deliberately qualitative outcomes. “Observed working” means a team member confirmed the behavior in the application. It does not mean that every input of that category always succeeds. A table cell is left unquantified when no count or timing was recorded.

Scenario	Expected result	Group observation
Invoice / receipt upload	File processed and proposed category shown	Most samples processed; occasional errors
Certificate / resume / PDF	Recognized text and fields available	Working overall; occasional inaccurate fields
Manual correction	Edited field and category can be saved	Confirmed working
Cabinet and search	Documents stored and retrievable	Confirmed working
Export	Available documents can be exported	Confirmed working

Table 6.2: Qualitative manual test scenarios; no numerical accuracy was recorded.

For client platforms, the confirmed environments were Flutter Web running locally in a browser and an Android emulator. No claim is made that the team tested the current build on a physical Android phone, on iOS hardware or on a public production site. Likewise, the optional document-sharing and administration flows were not clearly confirmed by the group, so they are not marked as passed manual tests.

Validation, Error Handling and Limitations

The code contains safeguards for cases that a user may not see during a successful demonstration. Before accepting an upload, the backend checks the extension, file signature and whether the PDF or image can be parsed. It limits accepted content according to configuration. For processing failures, the upload job records an error state and the interface provides a retry path. The result is designed to fail visibly rather than creating a broken record that appears complete.

The review decision flow is particularly important when AI output is imperfect. In the actual implementation, documents enter Needs Review, and the user confirms a corrected or accepted set of fields before the record becomes verified. OCR confidence, suggested sort confidence and verification are stored as separate notions. A test that simply sees “OCR 93%” on the screen does not prove that the extracted dates or names are 93% correct.

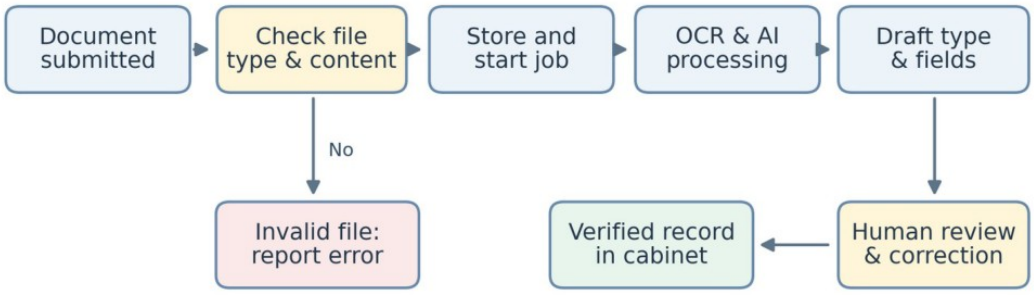


Figure 6.1: Validation and human review decision flow

CHAPTER 7: RESULTS & DISCUSSION

7.1 Performance Evaluation

The principal observed result is that DocSort completed a practical document-management sequence with the team's sample inputs. Documents could be submitted, processed with OCR and Gemini, reviewed, saved into a cabinet, searched and exported. This matters because the original project objective was not to produce OCR text in isolation. It was to turn digital files into accessible records while allowing the user to intervene when an automatic suggestion was wrong.

The team reported successful processing for most of the sample documents it tried, but no fixed number of documents or independent ground-truth labels were recorded. Some fields were missing or incorrect, and some documents were classified incorrectly. Manual corrections were possible. The correct performance conclusion is therefore qualitative: the pipeline worked for a range of tested examples but did not produce consistently correct extraction and classification without review.

DocSort achieved several specific outcomes. First, it joined a Flutter user interface to FastAPI so that a user action could trigger backend processing. Second, it combined a local OCR stage with an optional external AI stage while keeping the original file separate from extracted metadata. Third, it offered a review screen that helped the team recover from the kinds of field and category errors actually observed during testing. Finally, the cabinet, search, analytics and export features made the processed information available after the initial upload.

There are also signs of work still needed. A report-like PDF displayed in the Review screenshot contains extracted field labels that require human judgement. Some records in Search are shown as Unknown or Other. The Analytics screenshot contains a small bottom-overflow layout warning. Such details are relevant observations, not embarrassments to be concealed: they explain why the review stage and future UI refinements remain necessary.

7.2 Comparison with Existing Systems

DocSort is designed to solve a familiar personal file-management problem, but established products already offer parts of the same experience. A normal operating-system folder is excellent for keeping files but often relies on manual naming and placement. Document libraries such as SharePoint provide metadata-based management and collaboration capabilities [7]. Managed document AI products focus on services for understanding text and forms [8]. DocSort combines a modest subset of these ideas with an explicit user correction workflow in one student-developed application.

Criterion	Manual folders	Managed platforms	DocSort prototype
Document capture	Manual	Varies	Upload / mobile scan
OCR and extraction	Not built in	Product-dependent	Tesseract + optional Gemini
Human correction	Manual naming	Product-dependent	Review form
Organization	Folders only	Rich libraries	Cabinet and search
Production maturity	Local only	Established deployments	Academic prototype

Table 7.1: Feature-scope comparison, not a performance benchmark.

7.3 Observations

The most useful observation from the team's testing is that automation and correction need to be designed together. Tesseract and Gemini often produced useful initial information, but they also occasionally misread, omitted or misclassified details. The team did not have to abandon the workflow when that happened, because DocSort exposed the suggested fields for editing. A practical document manager should permit such corrections instead of asking users to accept opaque predictions.

A second observation concerns the application architecture. Returning a job identifier and presenting progress stages helped separate the short upload request from slower processing. Saving originals in private storage while retaining extracted fields in PostgreSQL made it possible to browse and search document records. These choices also introduce dependencies: the local backend needs to be running, and the cloud services must be reachable and configured for the full workflow to operate.

CHAPTER 8: CONCLUSION & FUTURE SCOPE

8.1 Summary of Work

DocSort was developed to make everyday digital documents easier to handle. The starting problem was simple: files accumulate quickly, their names rarely describe their contents, and manually sorting or searching through them takes unnecessary time. The team first explored a departmental application and later made personal document management the main goal. The final implementation demonstrates a Flutter interface connected to a FastAPI backend, private Supabase storage, PostgreSQL records, Tesseract OCR and optional Gemini-assisted document understanding.

8.2 Limitations

The first limitation concerns document understanding. OCR and AI can make mistakes on unclear scans, unusual layouts and long or mixed-format documents. The group observed missing or incorrect fields and unsuitable categories during testing. Human review makes the outcome correctable but does not eliminate the time and judgement needed to check important information. The application currently saves individual corrections without an identified mechanism for retraining its model automatically.

The second limitation is operational. Document processing uses FastAPI background tasks within the local server process. If that process stops at the wrong moment, work is not guaranteed to resume automatically. Rate limiting and some connection controls are process-local. The system has no independently verified production-scale throughput, shared distributed queue, automated backup/restore service or complete centralized observability. Such omissions are acceptable boundaries for a college prototype but would matter in a public deployment.

8.3 Future Enhancement

Further work could add a labelled evaluation dataset, a durable processing queue with recovery, better support for different screen sizes, and improved document extraction. Any feedback-learning mechanism would require consent and validation; it is not implemented in the current prototype.

REFERENCES / BIBLIOGRAPHY

[1] R. Smith, “An Overview of the Tesseract OCR Engine,” in Proc. 9th International Conference on Document Analysis and Recognition (ICDAR), 2007, pp. 629– 633. doi: 10.1109/ICDAR.2007.4376991.

[2] G. Jaume, H. K. Ekenel, and J.-P. Thiran, “FUNSD: A Dataset for Form Understanding in Noisy Scanned Documents,” in 2019 International Conference on Document Analysis and Recognition Workshops (ICDARW), 2019, pp. 1–6. doi: 10.1109/ICDARW.2019.10029.

[3] Y. Xu, M. Li, L. Cui, S. Huang, F. Wei, and M. Zhou, “LayoutLM: Pre-training of Text and Layout for Document Image Understanding,” in Proc. 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, 2020, pp. 1192–1200. doi: 10.1145/3394486.3403172.

[4] M. Li et al., “TrOCR: Transformer-Based Optical Character Recognition with Pre-trained Models,” in Proc. AAAI Conference on Artificial Intelligence, vol. 37, no. 11, 2023, pp. 13094–13102. doi: 10.1609/aaai.v37i11.26538.

[5] G. Kim et al., “OCR-Free Document Understanding Transformer,” in Computer Vision – ECCV 2022, 2022. doi: 10.1007/978-3-031-19815-1_29.

[6] J. Fung et al., “Human-in-the-loop Technical Document Annotation: Developing and Validating a System to Provide Machine-Assistance for Domain-Specific Text Analysis,” NIST Technical Note 2287, 2024. doi: 10.6028/NIST.TN.2287.

[7] Microsoft, “Use metadata to find content in document libraries,” Microsoft Learn. Available: https://learn.microsoft.com/en-us/microsoft-365/documentprocessing/metadata-search (accessed Oct. 2026).

[8] Google Cloud, “Document AI,” product documentation. Available: https://cloud.google.com/document-ai (accessed Oct. 2026).

REFERENCES / BIBLIOGRAPHY (CONTINUED)

[9] M. Jones, J. Bradley, and N. Sakimura, “JSON Web Token (JWT),” RFC 7519, Internet Engineering Task Force, May 2015. Available: https://www.rfc-editor.org/rfc/rfc7519.html.

[10] OWASP Foundation, “Password Storage Cheat Sheet,” OWASP Cheat Sheet Series. Available: https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html (accessed Oct. 2026).

[11] Flutter Development Team, “Build for and integrate with multiple platforms,” Flutter documentation. Available: https://docs.flutter.dev/platform-integration (accessed Oct. 2026).

[12] Supabase, “Storage Buckets: Public and Private Buckets,” official documentation. Available: https://supabase.com/docs/guides/storage/buckets/fundamentals (accessed Oct. 2026).

[13] FastAPI, “Background Tasks,” official documentation. Available: https://fastapi.tiangolo.com/tutorial/background-tasks/ (accessed Oct. 2026).

[14] Google AI for Developers, “Structured outputs,” Gemini API documentation. Available: https://ai.google.dev/gemini-api/docs/structured-output (accessed Oct. 2026).

[15] DocSort Project Team, “DocSort 1.0.0 Source Repository and Project Documentation,” GitHub, 2026. Available: https://github.com/N0tDeb/docsort (accessed 30 Oct. 2026).

[16] D. Kanjalkar, A. Thombre, P. Joshi, and R. More, “DocSort: A Review-Centered Workflow for OCR-Assisted Document Management,” submitted to the Asian Conference on Confluence in Technology (ACCT 2027), Microsoft CMT Paper ID 502, 30 Oct. 2026 (decision pending).

The primary source for the implementation descriptions in Chapters 3–8 is the team’s project repository [15]. External references provide background rather than performance evidence for DocSort. The team’s related paper [16] has been submitted; neither acceptance nor publication is claimed.

APPENDICES

Appendix A: Source Files and Verification Material

The development archive contains the Flutter client, Python FastAPI backend, SQL schema, tests and internal project notes. Its principal folders are summarized below; this is not a complete listing of every dependency or build artifact.

Source-code repository (GitHub): https://github.com/N0tDeb/docsort

Location in repository	Main contents
docsort/lib/	Flutter screens, features, network and routing
docsort-backend/app/api/	Authentication, uploads, search and document routes
docsort-backend/app/services/	OCR, classification, filing and other services
docsort-backend/schema.sql	PostgreSQL table definitions and relationships
docsort-backend/tests/	Backend automated validation
docs/	Workflow, evaluation and release notes (FLOW.md, EVALUATION.md, FINAL_YEAR_RELEASE.md)

The complete source code is available through the repository link above; this appendix provides a concise map of its main folders rather than reproducing the full codebase.

The screenshots in Chapters 4 and 5 were captured by the project group from a local Flutter Web session using mobile-device emulation. The source archive and screenshots provide the basis for the implementation descriptions in this report; the study did not include a new quantitative benchmark.

The related DocSort paper is identified in Appendix B, with CMT submission evidence reproduced in Appendix C. Conference acceptance and presentation certificates have not been issued.

Appendix B: Related Research Manuscript

Paper: DocSort: A Review-Centered Workflow for OCR-Assisted Document Management

Authors: Debashish Kanjalkar, Abhay Thombre, Prajwal Joshi, and Rahul More

Project Mentor: Dr. Jaykumar Shreeram Dhage

Submitted to: Asian Conference on Confluence in Technology (ACCT 2027); Microsoft CMT Paper ID 502; 30 Oct. 2026 at 21:34 IST.

Companion PDF: DocSort_ACCT2027_Submitted_Paper_ID_502.pdf (6 pages, supplied separately). Status: Submitted; decision pending. No acceptance, presentation or publication is claimed.

Appendix C: Conference Submission Confirmation

DocSort was submitted to the Asian Conference on Confluence in Technology (ACCT 2027) through Microsoft CMT. The submission summary records Paper ID 502 and creation on 30 October 2026 at 21:34:36 IST. The listed authors are Prajwal Joshi, Abhay Thombre, Debashish Kanjalkar and Rahul More. The submitted PDF is named “DocSort Reseach Paper.pdf” in the CMT record (spelling as shown in the confirmation).

Status at the time of this report: submission recorded; acceptance decision pending. No acceptance, presentation or publication certificate has been received. The excerpts below reproduce the submission details in the two-page CMT Submission Summary; the original PDF (which also contains author email addresses) is supplied separately.

CMT submission record: conference, paper ID and title (page 1).

Submission Summary

Conference Name
Asian Conference on Confluence in Technology

Paper ID
502

Paper Title
DocSort: A Review-Centered Workflow for OCR-Assisted Document Management

CMT submission creation timestamp (page 1).

Created
2026-10-30 21:34:36 IST

CMT submitted-file entry (page 2).

Submission Files
DocSort Reseach Paper.pdf (263.5 Kb, 2026-10-30 21:34:28 IST)